TripEase

Sentiment-Aware AI Agent for

Intelligent NYC Travel Itinerary Planning

NLP Parsing | ABSA | RAG | VRP Solver | Conversational Agent

Rupeet Kaur (rk3408) | EECSE6895 Final Project

Goal & Motivation

Build an AI agent that converts natural language trip queries into fully optimized, time-aware NYC itineraries - with zero hallucination at any stage.

The Problem with Existing Tools

- Generic tools ignore real visitor sentiment
- LLMs hallucinate POI hours and travel times
- Opening hours, day capacity, routes ignored
- No structured reasoning behind POI selection

Example Query → What Should Happen?

"3-day family trip. Museums and parks.
Empire State is a must.
Prefer mornings. Don't rush us."

1. Parse constraints + implicit preferences
2. Score POIs via real sentiment analysis
3. Route globally with hard-constraint solver
4. Schedule exact times with violation checks

What's Novel — Key Challenges Addressed

01 Aspect-Based Sentiment Analysis

Scores each POI on user-relevant aspects (family-friendliness, crowds, exhibits) from real Google reviews — not just overall star ratings.

02 RAG over Itinerary History

Retrieves semantically similar past trips to adapt proven solutions and explain what informed the current recommendation to the user.

03 Global CVRPTW Optimization

OR-Tools solves assignment + sequencing + travel simultaneously across all days — no greedy per-day clustering that creates suboptimal routes.

04 Zero-Hallucination Architecture

LLM handles only language and answers follow-up queries. All scheduling, routing, and constraint satisfaction is handled entirely by deterministic components.

Challenges:

- LLMs hallucinate schedules
- No labelled travel ABSA dataset
- Implicit constraint parsing
- CVRPTW is NP-hard
- M1 memory budget

Data — NYC POI Database

Primary Database

Source

Google Places API (curated snapshot)

Files

nyc_pois_database.json + nyc_pois_by_category.json

Zones

Lower Manhattan · Midtown · UES · UWS · Brooklyn · Bronx · Queens · Staten Island

Categories

Tourism · History/Culture/Art · Parks/Nature · Nightlife · Recreation

Fields/POI

id · displayName · primaryType · categories · location · zone · rating · userRatingCount · businessStatus · regularOpeningHours · photos · reviews

Difficulties

Stale hours/status · Review noise · Sparse reviews on smaller POIs

Volume

Hundreds of POIs each with multiple free-text reviews. Sentiment analysis performed at scale on unstructured review text.

Variety

Structured metadata (hours, coordinates, ratings) + unstructured review text + geospatial data — all in one pipeline.

Veracity

Hours and status can be stale. ABSA neutralizes review noise. businessStatus = PERMANENTLY_CLOSED hard-filtered.

System Architecture, Technology & Methodology

Stage	Module	Technology	Algorithm / Approach
Stage 1	NLP Query Parser	Llama-3.2-3B-Instruct (MLX 4-bit)	LLM → structured JSON constraint extraction
Stage 2	ABSA Sentiment Analysis	DeBERTa-v3-base-absa-v1.1 (yangheng)	Aspect-level sentiment on POI reviews
Stage 3	RAG Retrieval	Sentence-Transformers + FAISS	Semantic embedding + cosine similarity retrieval
Stage 4	Filter + Travel Matrix	Python + Haversine formula	POI hard filtering + travel time matrix
Stage 5	VRP Solver	Google OR-Tools (CVRPTW)	Global multi-day routing + POI assignment
Stage 6	Temporal Scheduling	Deterministic temporal walk	Exact clock-time assignment + violation flags
Stage 7	Conversational Agent	Llama-3.2-3B + collections.deque	Conversational QA over fixed session memory
Stage 8	UI & Visualization	Gradio Blocks + Folium + Plotly	Web interface + interactive map + timeline

All models run locally on M1 Pro 16 GB · No external API calls · No hallucination from network dependencies

Logical Codebase Structure

pipeline.py — Orchestrator

Stage 1	Stage 2	Stage 3	Stage 4	Stage 5	Stage 6
query_parser.py	ABSA.py	RAG.py	time_matrix.py	assignment sequence.py	Temporal Scheduling.py

Stage 7	Stage 8	Eval
Agent.py	UI.py	evaluation.py

SessionMemory — Single source of truth for all agent handlers. Saved once after Stage 6 completes. Pipeline never re-runs during conversation.

Key Principle: LLM handles language, answers follow-up queries · Solver handles logic · SessionMemory prevents redundant computation · All agent answers are grounded in pipeline output

End-to-End Demo — Stages 1 · 2 · 3: Perception & Retrieval

Input: "3-day family trip. Museums + parks. Empire State is a must. Prefer mornings. Don't rush us."

Stage 1 — NLP Query Parser

Llama-3.2-3B (MLX 4-bit)

trip_duration_days:
3

group_type:
"family"

pace:
"relaxed" → max 3 POIs/day

preferred_categories:
["history_culture_art", "parks_nature"]

must_include:
["Empire State Building"]

time_preferences:
{ morning: ["museum", "historical_place"] }

Stage 2 — ABSA Sentiment

DeBERTa-v3-base-absa-v1.1

POI analysed:
American Museum of Natural History

family-friendliness:
POSITIVE ↑

crowds:
NEGATIVE ↓

exhibits:
POSITIVE ↑

→ Effect:
Composite score boosts ranking for family query

Fallback:
Uses overall rating if reviews are insufficient

Stage 3 — RAG Retrieval

Sentence-Transformers + FAISS

Query embedded:
cosine similarity search over history

Retrieved:
3 past itineraries (similarity > 0.82)

Top match:
"3-day family, museums+parks, midtown"

→ Informs:
Candidate POI pool for Stage 4

→ Agent use:
Explains past trip relevance to user

RAG active?:
Stored in SessionMemory (rag_active flag)

End-to-End Demo — Stages 4 · 5: Filtering & Optimization

Stage 4 — Pre-processing & Travel Matrix

- ⊘ Hard-filter POIs closed on ALL assigned trip days
- ✓ Enforce must_include POIs survive filtering
- ⊞ Haversine travel time matrix between all POI pairs
-  Add zone centroid depot as start/end anchor each day

Output:
selected_pois · travel_time_matrix · depot · removed_pois

Stage 5 — CVRPTW Solver (OR-Tools)

Solves simultaneously:

- → Which POIs go on which day (assignment)
- → Order within each day (sequence)
- → Minimizes total travel globally — not per-day greedy

Hard Constraints

- ✓ Each POI visited exactly once
- ✓ Time windows (opening hours)
- ✓ Day capacity ≤ 12 hrs
- ✓ Max POIs/day by pace setting

Soft Penalties

- ~ Balance POIs across days
- ~ Group preferred zones
- ~ Time-of-day preferences

Output:
{ day_1: [POI_A, POI_B, POI_C], day_2: [...], day_3: [...] }

End-to-End Demo — Stages 6 · 7: Scheduling & Agent Response

Stage 6 — Temporal Scheduling

Deterministic walk through each day:

- 09:00 Start from depot (hotel)
- Travel 12 min → Arrive POI_A at 09:12
- Visit POI_A (90 min) → Depart 10:42
- Travel 18 min → Arrive POI_B at 11:00
- After 12:00 → Insert 60 min meal break
- Verify visit fits within opening hours
- Flag any violations with reason

Output per stop:

{ poi, start_time: '09:12', end_time: '10:42', travel_to_next: 18 }

day_summaries: total_travel_min · pois_visited · violations

Stage 7 — Conversational Agent

On itinerary generation — 3 blocks rendered together:

1. Day-by-day text schedule (exact clock times)
2. Folium map + Plotly Gantt timeline
3. "Trips similar to yours" (RAG past itineraries)

Follow-up Q&A handlers (no pipeline re-run):

Itinerary QA ← reads: itinerary + day summaries

POI Recommendations ← reads: Stage 5 dropped pois

Constraint Conflict ← reads: Stage 5 model data + Stage 6 summaries

Dropped POI Explanation ← reads: Stage 4 removed pois + Stage 5 data

Stage 8 — UI & Visualization (Gradio Blocks)

Chat Panel

→ User types query → generate itinerary fires (pipeline S1–S6)

→ 3 blocks rendered: schedule + map + RAG past trips

→ Follow-up chat → handlers read from SessionMemory only

→ Infeasible result? → Agent suggests constraint relaxation

Folium Interactive Map

Color-coded day markers · Directed polylines (depot→POIs→depot) · Popup: name, schedule, zone, violation flags · Photo thumbnails (onerror fallback)

Plotly Gantt Timeline

Per-day schedule bars · Hover: POI name, rating, ABSA score · Meal break blocks visible · Travel gaps shown

RAG "Similar Trips" Block

Similarity score, matched profile, key POI names. Re-renders only on new itinerary generation — not on every chat turn.

Design Rule: Map and timeline re-render ONLY when a new itinerary is generated — not on every chat message. Keeps interaction fast and avoids visual confusion.

Folium Map Features

- Numbered markers per stop (colour = day)
- Directed polylines: depot → POIs → depot
- Popup: name · time range · zone · type
- Violation flags shown in red in popup

Plotly Timeline Features

- Gantt-style per-day schedule bars
- Hover: POI name + rating + ABSA score
- Meal breaks as separate labelled blocks
- Travel gaps visible between POI bars

Design Decisions

Decision	Choice	Rationale
LLM for scheduling?	**No — OR-Tools only**	LLMs hallucinate times and opening hours
LangChain memory?	**No — collections.deque**	LangChain unavailable in project environment
Re-run pipeline per Q&A?	**No — SessionMemory**	Pipeline is expensive; answers are deterministic
RAG at query time?	**Yes — FAISS retrieval**	Fast; no re-embedding needed at runtime
OSRM travel times?	**Haversine for now**	M1 16 GB budget; OSRM is a production upgrade path
Per-day greedy routing?	**No — global CVRPTW**	Greedy creates local optima that are globally suboptimal
Streamlit vs Gradio?	**Gradio Blocks**	Simpler chatbot + multi-panel state management
History scope in agent?	**Q&A turns only**	Full itinerary excluded from deque — too large for context window

Key Principle: LLM handles language · Solver handles logic · SessionMemory is the single source of truth

Evaluation & Results — TripEase Agent vs. ChatGPT

Same query given to both. ChatGPT itinerary manually transcribed to CSV → run through evaluation on identical metrics.

Metric	TripEase Agent	ChatGPT
Total travel time (min)	336.00	494.00
Backtracking score (%)	11.11	12.50
Cross-zone rate (%)	16.67	88.45
Day balance std	0.00	0.47
Time pref adherence (%)	100.00	100.00
Invalid day assignment (%)	16.00	55.53
Opening hours adherence (%)	100.00	100.00

Key Findings

16% invalid assignments(ChatGPT 55%) — hard constraint guarantee

100% hours adherence — solver enforces time windows

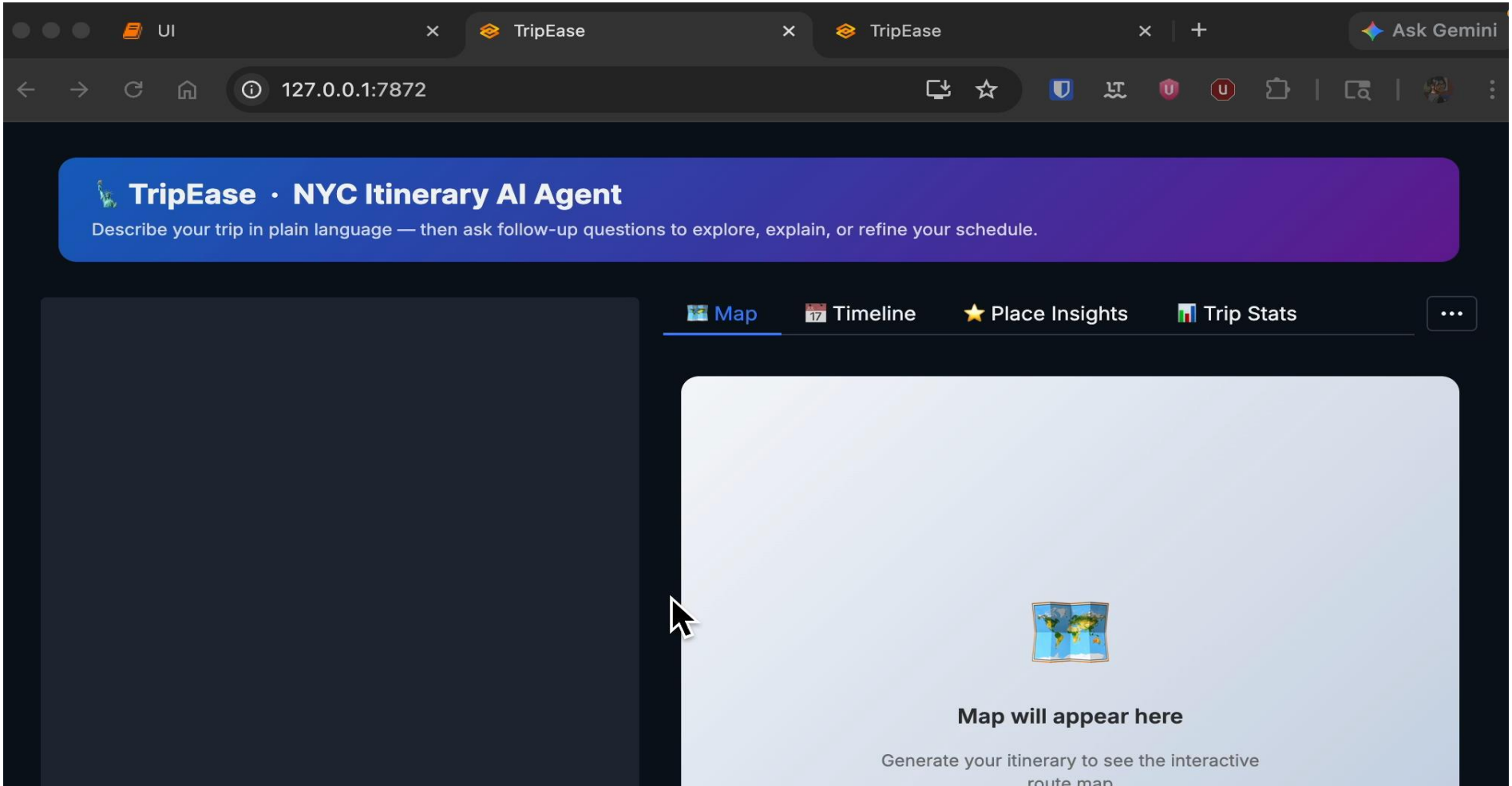
Lower travel time — global CVRPTW vs local suggestions

ChatGPT: fluent but often violates scheduling constraints

Results - TripEase Agent vs. ChatGPT Itinerary Map Comparison

```
Zone change: Prev id ChIJ9U1mz_5YwokRosza1aAk0jM, zone midtown | Current id ChIJaWjW_PFYwokRFD8a2YQu12U, zone upper_east_side
Zone change: Prev id ChIJaWjW_PFYwokRFD8a2YQu12U, zone upper_east_side | Current id ChIJB3rTagBZwokRj0LRmWkZ9eo, zone midtown
Zone change: Prev id ChIJCXoPsPRYwokRsV1MYnKBfaI, zone upper_east_side | Current id ChIJF2QfxY_0wokR8bwM-YbJ2aU, zone bronx
Zone change: Prev id ChIJaXQRs6lZwokRY6EFpJnhNNE, zone midtown | Current id ChIJ4zGFAZpYwokRGUGph3Mf37k, zone upper_east_side
Zone change: Prev id ChIJ4zGFAZpYwokRGUGph3Mf37k, zone upper_east_side | Current id ChIJt3n6V0NYwokR_v_9QL79BAY, zone upper_west_side
```

Demo Video


UI
TripEase
TripEase
Ask Gemini
127.0.0.1:7872
TripEase · NYC Itinerary AI Agent
Describe your trip in plain language — then ask follow-up questions to explore, explain, or refine your schedule.
Map
Timeline
Place Insights
Trip Stats
Map will appear here
Generate your itinerary to see the interactive

Limitations & Future Work

Current Limitations

Haversine Travel Time
Ignores subway routes, traffic, and one-way streets. Can misestimate inner-borough trips.

Static POI Database
Events, temporary closures, seasonal hours not reflected without a live API refresh.

Generic ABSA Model
Not fine-tuned on travel aspects. May miss nuances like 'wait time' or 'ticketing hassle'.

LLM Parsing Errors
Ambiguous queries can cause Stage 1 mis-extractions. No interactive clarification loop yet.

Small Itinerary History
RAG retrieval quality improves with volume — course-project scale limits diversity.

Future Work

OSRM Real Transit Times
Replace Haversine with actual routing for production-grade travel estimates including subway.

Live POI Status Refresh
Periodic Google Places API sync for up-to-date hours, status, and newly opened venues.

Fine-tuned Travel ABSA
Fine-tune DeBERTa on travel corpus for: wait time, accessibility, kid-friendliness, value.

Multi-modal Awareness
Incorporate weather forecasts and time-sensitive events into day planning decisions.

Generalise Beyond NYC
Pipeline is city-agnostic — POI database swap enables deployment in any major city.

TripEase demonstrates that **structured AI pipelines with hard-constraint solvers** outperform pure LLMs for optimization-heavy planning tasks.